

“FraudShield: Unified Fraud Detection & Risk Scoring Platform”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

D. ROHINI TANMAYI :2520030349

S. MANVI VAISHNAVI :2520030326

VARSHITH :2520030598

Under the Esteemed Guidance of

Dr Yerragudipadu Subbarayudu

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“FraudShield: Unified Fraud Detection & Risk Scoring Platform”** is a record of Bonafide work of **D. ROHINI TANMAYI - 2520030349, S. MANVI VAISHNAVI - 2520030326, VARSHITH - 2520030598** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

D. ROHINI TANMAYI – 2520030349

S. MANVI VAISHNAVI – 2520030326

VARSHITH – 2520030598

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project based report entitled **“FraudShield: Unified Fraud Detection & Risk Scoring Platform”** is a bonafide work done and submitted by **D. ROHINI TANMAYI - 2520030349, S. MANVI VAISHNAVI - 2520030326, VARSHITH - 2520030598** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2025-2026**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation to our Course Coordinator **Dr Yerragudipadu Subbarayudu**, and our guide **Dr Yerragudipadu Subbarayudu**, Department of Computer Science, which cannot be expressed in words, for his tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Rao (CSE)/Dr. Kaja Shareef (CSIT)**, Head of the **Department for Computer Science Engineering/ Department for Computer Science and Information Technology** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff, and also those who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

D. ROHINI TANMAYI – 2520030349

S. MANVI VAISHNAVI – 2520030326

VARSHITH – 2520030598

Abstract

Digital payments, mobile banking and e-commerce have grown at an extraordinary pace, and with that growth the volume and sophistication of financial fraud has increased as well. Account takeover, card-not-present fraud, synthetic identities and rapid-fire velocity abuse now move faster than the static, rule-based thresholds that many institutions still depend on. These legacy rules are slow to update, generate large numbers of false positives that block genuine customers, and rarely explain why a transaction was stopped. Fraud teams are further slowed down by disconnected tools for monitoring, alerting and investigation.

To tackle this, our project focuses on the design and development of **FraudShield: a Unified Fraud Detection and Risk Scoring Platform** that combines machine learning with real-time stream processing. Transactions are ingested through an Apache Kafka streaming pipeline, enriched with behavioural features such as transaction velocity in the last five minutes, distance from the customer's home location, account age, hour of day, new-device and new-payee indicators, and channel. These features feed an XGBoost classifier tuned for highly imbalanced fraud data, which produces a 0–100 risk score for every transaction.

A FastAPI service performs low-latency inference, Redis keeps hot behavioural counters and the live feed in memory, and PostgreSQL stores durable, queryable records of transactions, scores, alerts and investigation cases. SHAP-based explainability converts every prediction into a clear list of contributing risk factors, so analysts can see exactly *why* a transaction was flagged.

All of this is brought together in the **Fraud Risk Command Center**, a web dashboard with seven views: Dashboard, Live Monitoring, Transactions, Alerts, Cases, Score Transaction and Models. It presents KPIs, risk-score distributions, fraud trends, top risk factors, a live transaction feed, alerts by channel and severity, a full case-management workflow, an interactive form for scoring new transactions, and model performance with feature importances. The system is evaluated on precision, recall, F1-score and AUC-ROC. In summary, FraudShield delivers a complete, industry-style pipeline from real-time ingestion to explainable, analyst-facing decisions in a single unified platform.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	5
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	8-13
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	14-16
3	Technology Stack	3.1 Front-end (HTML5, CSS3, JavaScript, Chart.js) 3.2 Back-end (Python, FastAPI, Kafka) 3.3 Database (PostgreSQL, Redis) 3.4 Version Control (Git, GitHub) 3.5 ML libraries and external services	17-21
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (e.g., cloud, local, CI/CD pipelines)	22-25
5	Design	5.1 ER Diagram / Database schema 5.2 Risk scoring and case workflow design	26-28
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	29-33
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	34-37
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	38-41
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	42-44
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	45-47
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	48-49

S. No.	Chapters	Topics	Page.no
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	50-51
13	References	- Books, research papers, documentation sites used	52-53
14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide -Geo Tag photos with guide -Review forms with guide signatures	54-60

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

The way people pay for goods and services has changed more in the last decade than in the previous half-century. Unified payment interfaces, mobile wallets, internet banking, card-on-file e-commerce checkouts and API-driven merchant integrations have made it possible to move money in seconds from almost anywhere. Financial institutions, payment gateways and e-commerce platforms now process millions of transactions every day, and customers expect each of them to be approved instantly and without friction.

This convenience has a cost. Every new channel through which money can move is also a new channel through which it can be stolen. Fraud has become organised, automated and fast. Common patterns include **account takeover**, where stolen credentials are used from a new device; **card-not-present fraud**, where card details are used online without the physical card; **synthetic identities**, where fraudsters build fake profiles that age quietly before being used; and **velocity-based abuse**, where many small transactions are fired in a short window to test or drain an account before anyone notices. These tactics evolve constantly, often within weeks.

Traditionally, banks and payment providers relied on rule engines to catch such activity: *flag if amount exceeds a fixed limit, block if the country differs from the billing address*, and so on. Rules are easy to understand but rigid. A fixed threshold that is low enough to catch fraud blocks large numbers of genuine customers, while a threshold high enough to avoid annoying customers lets clever fraud slip through. Rules are written and updated by hand, so by the time a new rule reaches production the fraud pattern it targets has frequently moved on.

Machine learning combined with real-time stream processing has therefore become the industry standard approach. Instead of a single threshold, a model learns how combinations of signals — a large amount, at an unusual hour, from a new device, to a new payee, far from the customer's home, shortly after several other transactions — relate to the likelihood of fraud. Streaming platforms such as Apache Kafka allow each transaction to be evaluated the moment it happens rather than in an overnight batch.

However, a model on its own does not solve the operational problem. Fraud analysts still need to see what is happening right now, understand why a particular transaction was flagged, triage alerts by severity, and track investigations through to a decision. In many organisations, these tasks are spread across disconnected tools: one for monitoring, another for alerts, a spreadsheet for case notes, and no explanation at all for the model's decision.

The idea for this project emerged from observing these gaps. **FraudShield** is designed as a single, unified platform that demonstrates the complete fraud-detection lifecycle end to end: data ingestion from a stream, behavioural feature engineering, imbalance-aware model training, real-time risk scoring, SHAP-based explanation, alerting, case management, and an analyst-facing dashboard called the **Fraud Risk Command Center**.

From an academic perspective, the project brings together several areas of computer science that are usually studied separately: machine learning and model evaluation, distributed messaging, REST API design, relational and in-memory databases, and front-end data visualisation. Building FraudShield required these pieces to work together under realistic constraints such as low latency, class imbalance, and the need for human-readable explanations.

1.2 Problem Statement

Despite the progress in digital payments, fraud detection in many environments still depends on static, manually maintained rules and on a patchwork of tools that do not talk to each other. This leads to four closely related problems that FraudShield sets out to address.

1. Rigid, static detection logic

Rule-based systems flag transactions using fixed thresholds such as a maximum amount. They cannot weigh several weak signals together, and they cannot adapt to new fraud patterns without someone rewriting the rules. Fraudsters quickly learn to stay just below known thresholds.

2. High false-positive rates

Because rules are blunt, they block many genuine transactions. Every false positive is a frustrated customer, a support call, and potentially a lost sale. At the same time, sophisticated fraud that does not trip any single rule passes through undetected.

3. No explanation for decisions

When a transaction is blocked, analysts often cannot see which condition fired or how strongly each factor contributed. Machine-learning models make this worse if they are treated as black boxes. Without explanations, analysts cannot confidently confirm or overturn a decision, and the organisation cannot justify its actions to customers or regulators.

4. Fragmented monitoring, alerting and investigation

Real-time monitoring, alert triage and case investigation are frequently handled in separate systems. Analysts switch between screens, copy transaction IDs by hand, and lose track of which alerts have been reviewed, escalated or closed.

These problems can be summarised as four guiding questions:

1. How can suspicious transactions be identified accurately and in real time, without relying on rigid, static rules?
2. How can false positives be reduced while still catching sophisticated, evolving fraud patterns?
3. How can fraud analysts be given clear, explainable reasons for every flagged transaction?
4. How can monitoring, alerting and case management be unified into a single platform?

FraudShield answers these questions by combining a streaming pipeline, an imbalance-aware XGBoost model that produces a 0–100 risk score, SHAP-based explanations, and a unified command-center dashboard.

1.3 Scope of the Project

The scope of FraudShield covers the full path of a transaction from the moment it is created to the moment an analyst closes the related case. The project is built as a working, demonstrable system rather than a purely theoretical model, and it is designed so that each component could be replaced with a production-grade equivalent later.

In scope

- **Transaction ingestion:** a Kafka-based streaming pipeline that receives transaction events from a simulator representing Mobile, Web, API and POS channels.
- **Feature engineering:** computation of behavioural features — amount, distance from home (km), velocity in the last 5 minutes, account age (days), hour of day, new device, new payee and channel.
- **Model training:** an XGBoost classifier tuned for class-imbalanced fraud data, compared against a RandomForest baseline, with evaluation on precision, recall, F1-score and AUC-ROC.
- **Real-time scoring:** a FastAPI service that converts model probability into a 0–100 risk score and a Low / Medium / High risk level.
- **Explainability:** SHAP-based feature contributions translated into a human-readable “Why was this flagged?” list and a score-breakdown table.
- **Data layer:** Redis for in-memory counters, caching and the live feed; PostgreSQL for durable storage of transactions, scores, alerts and cases.
- **Fraud Risk Command Center:** a web dashboard with Dashboard, Live Monitoring, Transactions, Alerts, Cases, Score Transaction and Models views.

- **Case management:** cases with priority, status (Open, In Review, Escalated, Closed) and assigned analyst.
- **Demo controls:** the ability to reset and re-seed the database with fresh sample transactions before a live presentation.

Out of scope

- Connection to real banking networks or live card data; all transactions are simulated or come from public/synthetic datasets.
- Regulatory reporting (for example, suspicious activity report filing) and full KYC/AML workflows.
- Automatic blocking of payments in an external system; FraudShield scores and flags, and humans decide.
- Production-scale hardening such as multi-region deployment, which is discussed under future enhancements.

This scope is wide enough to demonstrate an industry-style architecture while remaining achievable for a three-member team within one academic year.

1.4 Objectives of the Project

The primary objective of FraudShield is to show that accurate, real-time and explainable fraud detection can be delivered through a single unified platform. The specific objectives are:

1. **Build a feature-engineered, imbalance-aware ML model** that reliably flags fraudulent transactions. Fraud is typically a very small fraction of all transactions, so the model must be trained with techniques such as class weighting and resampling rather than plain accuracy.
2. **Design a low-latency pipeline** (Kafka → FastAPI → Redis/PostgreSQL) capable of scoring each transaction in real time, so that decisions can be made before the payment completes.
3. **Provide a Fraud Risk Command Center dashboard** for live monitoring and case triage, giving analysts KPIs, trends, a live feed, alerts and case management in one place.
4. **Make model decisions explainable** to non-technical fraud analysts using SHAP, so that every flagged transaction comes with clear, ranked reasons.
5. **Evaluate the system** rigorously on precision, recall, F1-score and AUC-ROC rather than on accuracy alone, which is misleading for imbalanced data.

6. **Unify alerting and investigation** through a case-management workflow with priority, status and assignment.

Together, these objectives turn a machine-learning model into a usable operational tool.

1.5 Importance of the Application

Fraud has a direct financial cost, but its indirect costs are often larger: chargeback fees, operational effort, regulatory penalties, and above all, loss of customer trust. A platform like FraudShield matters for several reasons.

Financial protection

Catching a fraudulent transaction before it settles is far cheaper than recovering funds afterwards. Real-time scoring allows suspicious payments to be held for review while genuine payments proceed normally.

Better customer experience

A model that weighs many signals together can be both stricter on real fraud and more lenient on genuine but unusual behaviour. Reducing false positives means fewer declined cards, fewer embarrassing moments at checkout, and fewer calls to customer support.

Transparency and accountability

Explainability is increasingly expected by regulators and by customers. When FraudShield flags a transaction, it shows that, for example, the amount was unusually high, the device was new, and the transaction occurred 1,200 km from the customer's home at midnight. This allows analysts to justify actions and to spot cases where the model may be wrong.

Analyst productivity

By bringing monitoring, alerts and cases into one command center, analysts spend less time switching tools and more time making decisions. Alerts are grouped by severity and channel, and each case carries its priority, status and owner.

Educational value

For students and educators, FraudShield is a compact, end-to-end reference that shows how modern data products are built: streaming ingestion, feature stores, model serving, explainability, databases and dashboards working together.

1.6 Target Users / Audience

FraudShield is designed around the people who detect, investigate and manage fraud, as well as those who build and study such systems.

User group	How they use FraudShield
Fraud analysts	Monitor the live feed, review alerts, open the transaction detail to read the risk breakdown, and progress cases from Open to Closed.
Fraud team leads / supervisors	Track KPIs, fraud trends and alert volumes by channel; assign and escalate cases; decide on investigation priorities.
Risk and compliance officers	Use SHAP explanations and case history to understand and justify decisions made on flagged transactions.
Data scientists / ML engineers	Inspect model performance and feature importances on the Models page; test hypothetical transactions using the Score Transaction form.
Payment gateways, fintechs and e-commerce platforms	Represent the organisations that would integrate the scoring API into their checkout or payment flows.
Students and educators	Use the project as a reference implementation of a real-time, explainable ML system.

By serving both operational users and technical users, FraudShield bridges the gap between building a fraud model and actually using it.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

FraudShield runs several services at the same time — Kafka, the FastAPI scoring service, Redis, PostgreSQL, the transaction simulator and the web dashboard. Model training is also more demanding than serving. The hardware requirements below were chosen so that the complete stack can run comfortably on a single developer laptop using Docker.

Table 2.1 Hardware requirements

Component	Minimum	Recommended
Processor	Intel Core i5 (8th gen) / AMD Ryzen 5, 4 cores	Intel Core i7 / AMD Ryzen 7, 8 cores
RAM	8 GB	16 GB (Kafka + PostgreSQL + model training together)
Storage	20 GB free (SSD)	50 GB free NVMe SSD
Network	Broadband for package installation	Stable connection for Docker image pulls and GitHub
Display	1366 × 768	1920 × 1080 for the full dashboard grid
Client device	Any modern laptop or desktop with a browser	Same

Processor. XGBoost training and SHAP value computation benefit from multiple cores; XGBoost parallelises tree construction across available threads. During serving, a single prediction is fast, so the processor mainly matters during training and bulk re-scoring.

Memory. Kafka and Zookeeper (or KRaft), PostgreSQL and Redis each reserve memory. With the dataset loaded into pandas during training, 8 GB is workable but 16 GB gives headroom for running the dashboard, IDE and browser at the same time.

Storage. An SSD significantly speeds up Docker image builds, PostgreSQL writes and loading of training data. Kafka log segments and PostgreSQL data volumes grow with demo traffic, so periodic resets using the Reset Demo Data control keep storage under control.

Client side. Analysts only need a modern browser. The dashboard is a lightweight web application whose charts are rendered by Chart.js on the client, so no special hardware is needed to use it.

2.2 Software Requirements

The software stack was selected to match what is used in industry for real-time ML systems while remaining free and open source.

Table 2.2 Software requirements

Category	Software / Version	Purpose
Operating system	Windows 10/11, Ubuntu 22.04 or macOS	Development and hosting environment
Programming languages	Python 3.10+, JavaScript (ES6+)	ML pipeline and backend in Python; dashboard logic in JavaScript
ML libraries	pandas, NumPy, scikit-learn, XGBoost, SHAP, imbalanced-learn	Data preparation, model training, evaluation, explainability, resampling
Backend	FastAPI, Uvicorn, SQLAlchemy, Pydantic	REST API, ASGI server, ORM and request validation
Streaming	Apache Kafka 3.x (kafka-python / confluent-kafka client)	Real-time transaction ingestion
Databases	PostgreSQL 14+, Redis 7+	Durable storage; in-memory counters, cache and live feed
Front-end	HTML5, CSS3, JavaScript, Chart.js	Fraud Risk Command Center dashboard and charts
Containerisation	Docker, Docker Compose	Running all services consistently
Browser	Chrome, Edge or Firefox (latest)	Accessing the dashboard

Python was chosen for the backend so that the same language and libraries are used for both training and serving. This avoids having to re-implement feature engineering in a second language, which is a common source of training/serving mismatch.

2.3 Development Tools and Frameworks

Table 2.3 Development tools

Tool	Use in the project
Visual Studio Code	Main editor for Python, JavaScript, HTML and CSS, with Python, Docker and REST client extensions.
Jupyter Notebook	Exploratory data analysis, feature experiments, model comparison and SHAP plots.

Tool	Use in the project
Git and GitHub	Version control, branching per module, pull requests and code review among the three team members.
Postman	Testing FastAPI endpoints such as scoring, transactions, alerts and cases; saving collections for regression tests.
Docker / Docker Compose	Running Kafka, PostgreSQL, Redis, the API and the web server together with one command.
pgAdmin / psql	Inspecting tables, verifying foreign keys and checking query performance in PostgreSQL.
RedisInsight / redis-cli	Inspecting velocity counters, cached scores and live-feed keys.
Swagger UI (FastAPI /docs)	Auto-generated, interactive API documentation used during integration.
PyTest	Unit tests for feature engineering, scoring logic and API endpoints.
Browser DevTools	Debugging dashboard rendering, network calls and Chart.js output.

Frameworks

FastAPI provides automatic request validation through Pydantic models, asynchronous request handling and built-in OpenAPI documentation. **XGBoost** provides gradient-boosted decision trees with native support for class weighting through the *scale_pos_weight* parameter. **SHAP** provides TreeExplainer, which computes exact Shapley values for tree models efficiently. **Chart.js** renders the donut charts, trend lines, horizontal bars, gauge and feature-contribution charts in the dashboard.

These tools allowed the team to work in parallel: the ML pipeline in notebooks and Python modules, the backend and data layer in FastAPI and Docker, and the dashboard in HTML, CSS and JavaScript, all integrated through well-defined JSON APIs.

CHAPTER-3 TECHNOLOGY STACK

3.1 Front-End Design

The front end of FraudShield is the **Fraud Risk Command Center**, a single-page web application built with HTML5, CSS3 and vanilla JavaScript (ES6+), with charts rendered by **Chart.js**. The page is organised as a fixed left sidebar and a main content area. The sidebar contains the FraudShield brand and seven navigation buttons; the main area contains a top bar and seven views, only one of which is visible at a time.

Layout structure

- **Sidebar:** brand logo, navigation items (Dashboard, Live Monitoring, Transactions, Alerts, Cases, Score Transaction, Models) and a version footer (v1.0.0).
- **Top bar:** page title, a search box for transactions, users and cases, a notification bell with a live alert-count badge, and the signed-in analyst's avatar and role.
- **Views:** each view is a `<section class="view">` element; switching views toggles the *active* class rather than reloading the page.

Why HTML, CSS and vanilla JavaScript

A framework-free front end keeps the dashboard light, fast to load and easy to run anywhere — it can be served as static files by Nginx or opened directly during a demo. Each view is populated by small JavaScript render functions that fetch JSON from the API and inject rows, cards and charts. CSS grid classes such as *kpi-grid*, *grid-2* and *grid-3* provide a responsive card layout, and colour classes (red, amber, green) are used consistently for High, Medium and Low risk.

Chart.js visualisations

Chart	Type	Location
Risk Score Distribution	Doughnut	Dashboard
Fraud Trend	Line	Dashboard
Alerts by Channel	Doughnut	Dashboard
Risk gauge (score / 100)	Half doughnut gauge	Live Monitoring
Model Explanation (Feature Contributions)	Horizontal bar	Live Monitoring
Feature Importances	Horizontal bar	Models

The Chart.js library is bundled locally (vendor/chart.umd.min.js) so that the dashboard works even without internet access during a presentation.

3.2 Back-End Design

The back end is written in Python and is made up of three cooperating parts: the streaming layer, the scoring service and the REST API.

Apache Kafka (streaming layer)

Kafka is a distributed, durable log that lets producers publish events and consumers read them at their own pace. In FraudShield, the transaction simulator publishes each transaction as a JSON event to the *transactions.raw* topic. A consumer reads these events, enriches them with behavioural features and passes them to the model; scored results are published to *transactions.scored*. Kafka decouples transaction producers from the scoring service, so bursts of traffic are buffered rather than lost.

FastAPI (scoring service and REST API)

FastAPI is a modern, high-performance Python web framework built on ASGI and served by Uvicorn. It was chosen because it validates requests automatically using type-annotated Pydantic models, supports asynchronous I/O for database and Redis calls, and generates interactive OpenAPI documentation at */docs*. The trained XGBoost model and SHAP explainer are loaded once when the service starts, so each request only performs feature preparation, prediction and explanation.

Machine-learning core

The model is an **XGBoost** gradient-boosted tree classifier. Gradient boosting builds trees one after another, each correcting the errors of the previous ones, and handles mixed numeric and binary features without scaling. Class imbalance is handled with *scale_pos_weight* and with resampling from **imbalanced-learn** during experiments. A **RandomForest** model from scikit-learn is trained as a baseline for comparison and its metrics are also reported on the dashboard.

SQLAlchemy

SQLAlchemy is used as the ORM between FastAPI and PostgreSQL. Tables are declared as Python classes, which keeps the schema in version control and makes queries for the Transactions, Alerts and Cases views straightforward and safe from SQL injection.

3.3 Database Design

FraudShield uses two databases with clearly separated responsibilities: Redis for speed and PostgreSQL for durability.

PostgreSQL 14+ (system of record)

PostgreSQL stores everything that must be kept and queried: users, devices, transactions, risk scores, risk factors (explanations), alerts, cases, analysts and model versions. A relational database fits this domain well because the entities are strongly related — a case belongs to an alert, an alert belongs to a transaction, a transaction belongs to a user — and these relationships must be enforced with foreign keys. Indexes on *created_at*, *user_id*, *risk_score* and case *status* keep dashboard queries fast.

Redis 7+ (in-memory layer)

Behavioural features such as *velocity_5min* need to be computed in milliseconds. Querying PostgreSQL for “how many transactions did this user make in the last five minutes” on every event would be slow. Instead, Redis keeps per-user sorted sets of recent transaction timestamps, sets of known devices and payees, and a short list of the most recent scored transactions for the live feed. Keys are given expiry times so memory use stays bounded.

Table 3.1 Redis key design

Key pattern	Type	Purpose
vel:{user_id}	Sorted set	Transaction timestamps; count members in last 300 s to get velocity_5min
dev:{user_id}	Set	Devices previously seen for this user (new_device check)
payee:{user_id}	Set	Payees previously paid by this user (new_payee check)
score:{txn_id}	Hash	Cached score and top factors for quick detail lookups
feed:live	List (capped)	Most recent scored transactions for the Live Transactions Feed
stats:alerts	Hash	Running counts of High/Medium/Low alerts for the bell badge

3.4 Version Control

Git was used for version control and GitHub as the shared remote repository. The repository is organised by module so that each team member could work independently with minimal merge conflicts.

```
fraudshield/
|-- ml/                                # notebooks, training scripts, saved model (D. Rohini Tanmayi)
|   |-- train.py
|   |-- features.py
|   |-- models/xgb_fraud.json
|-- backend/                           # FastAPI, Kafka consumer, DB models (S. Manvi Vaishnavi)
|   |-- app/main.py
|   |-- app/routers/                  # score, transactions, alerts, cases, model, demo
|   |-- app/db.py
|   |-- consumer.py
|   |-- simulator.py
|-- frontend/                          # Fraud Risk Command Center (Varshith)
|   |-- index.html
|   |-- style.css
|   |-- app.js
|   |-- vendor/chart.umd.min.js
|-- docker-compose.yml
`-- README.md
```

A simple branching strategy was followed: *main* always holds a working, demonstrable version; feature branches such as *feature/shap-explanations*, *feature/case-management* and *feature/kafka-consumer* were merged through pull requests after review by another team member. Commit messages were kept short and descriptive so that the history doubles as a log of progress.

3.5 ML Libraries and External Services

Table 3.2 Libraries and services

Library / service	Role in FraudShield
pandas / NumPy	Loading, cleaning and transforming transaction data; building feature matrices.
scikit-learn	Train/test splitting with stratification, RandomForest baseline, metrics (precision, recall, F1, ROC-AUC), confusion matrices.
XGBoost	Primary fraud classifier with scale_pos_weight for imbalance and early stopping on a validation set.
imbalanced-learn	SMOTE and random under-sampling experiments to compare imbalance strategies.
SHAP	TreeExplainer for per-transaction feature contributions and global feature importance.
Apache Kafka	Event streaming between simulator, feature/scoring consumer and downstream services.
Redis	Low-latency feature counters, caching and live feed.

Library / service	Role in FraudShield
PostgreSQL	Durable relational storage for all business entities.
Chart.js	Client-side charts in the dashboard.
Docker Hub images	Official images for Kafka, PostgreSQL, Redis and Nginx used in Docker Compose.

No paid third-party APIs are required. All components are open source, which keeps the project reproducible for anyone who clones the repository.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level Architecture Diagram

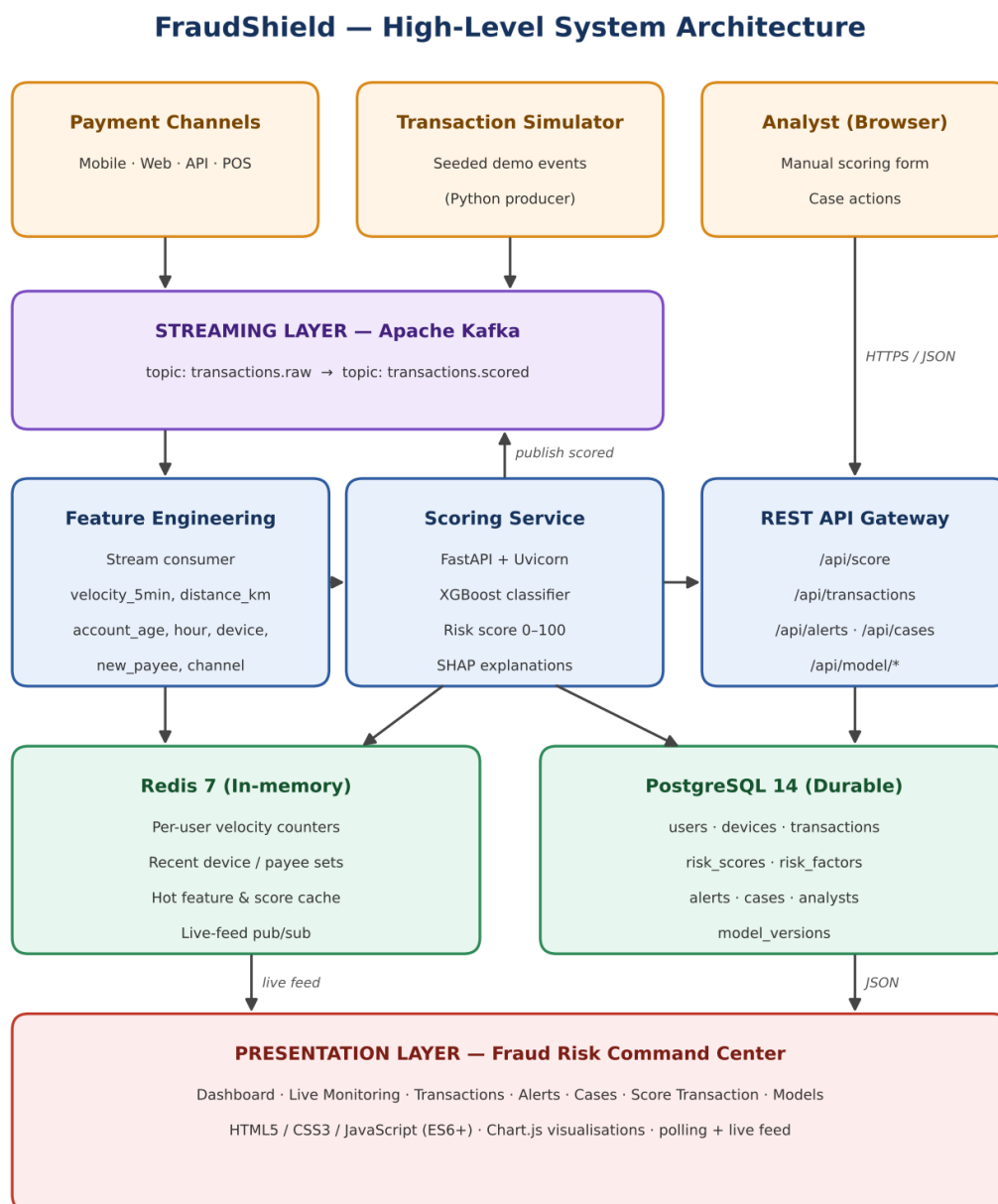


Figure 4.1 High-level architecture of FraudShield

FraudShield follows a layered, event-driven architecture. Transactions enter from payment channels (represented in the project by a transaction simulator), flow through Kafka, are enriched and scored, are stored in Redis and PostgreSQL, and are finally presented to analysts in the Fraud Risk Command Center. Analysts can also call the API directly from the dashboard, for example to score a hypothetical transaction or update a case.

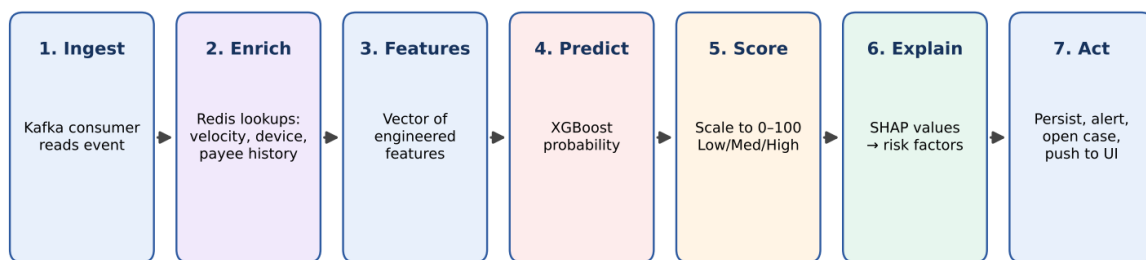


Figure 4.2 Real-time scoring pipeline for a single transaction

4.2 Description of Each Layer (Frontend, Backend, Database)

1. Source layer

The source layer represents where transactions come from. In a real deployment these would be card networks, UPI switches, mobile apps, web checkouts and point-of-sale terminals. In FraudShield a Python simulator generates realistic transactions across the Mobile, Web, API and POS channels, mixing ordinary behaviour with injected fraud patterns such as bursts of high-velocity payments, transactions far from home, and large payments from brand-new accounts and devices.

2. Streaming layer (Kafka)

Kafka receives every transaction as an event on the *transactions.raw* topic. Using a log rather than direct HTTP calls means the producer never waits for the model, traffic spikes are absorbed, and events can be replayed for testing. After scoring, results are published to *transactions.scored*, which allows other consumers (for example, a future notification service) to react without changing the scoring code.

3. Processing layer (feature engineering and scoring)

A Kafka consumer reads each raw event and builds the feature vector. Stateful features come from Redis: the consumer adds the transaction timestamp to the user's velocity set and counts entries in the last five minutes, checks whether the device and payee are already known, and computes distance from the stored home coordinates. The scoring service then runs the XGBoost model to obtain a fraud probability, scales it to a 0–100 risk score, assigns a risk level, and uses SHAP to compute each feature's contribution.

4. API layer (FastAPI)

The REST API exposes all data the dashboard needs and accepts analyst actions. It handles request validation, pagination, filtering (for example, cases by status) and error handling, and returns

consistent JSON responses.

5. Data layer (Redis and PostgreSQL)

Redis holds short-lived, frequently accessed state: velocity counters, device and payee sets, cached scores and the live feed. PostgreSQL holds the permanent record: every transaction, its score and explanation, any alert raised, the resulting case, and the model version that produced the score. Storing the model version with each score makes results auditable after the model is retrained.

6. Presentation layer (Fraud Risk Command Center)

The dashboard periodically polls the API and refreshes the live feed, KPIs and alert badge. Each of the seven views corresponds to a set of API calls, and all charts are drawn client-side with Chart.js. Because the presentation layer only consumes JSON, it could be replaced or extended (for example, by a mobile app) without touching the backend.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

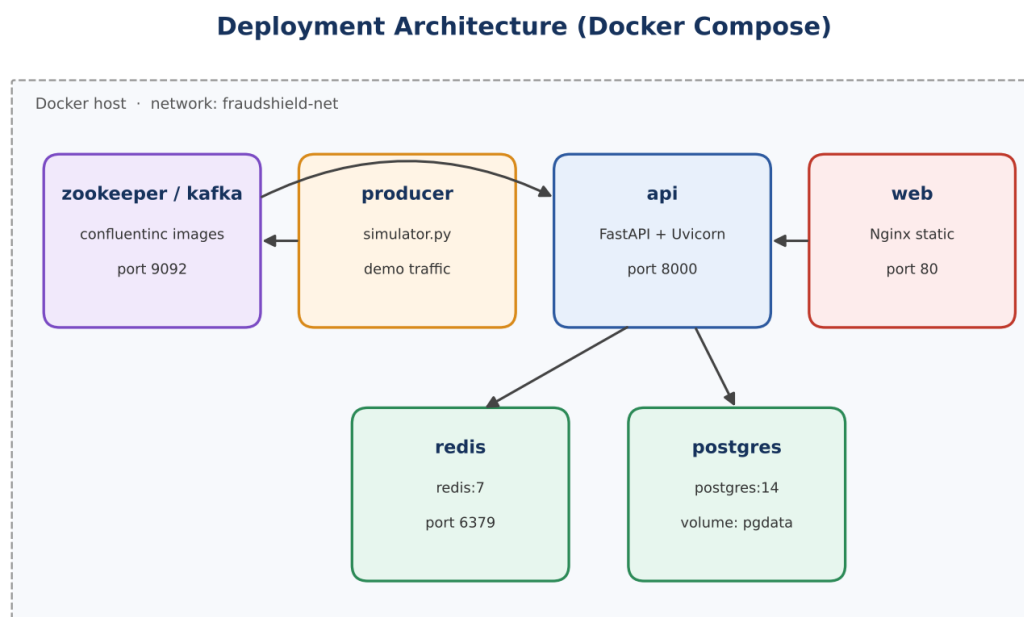


Figure 4.3 Container-based deployment using Docker Compose

For development and demonstration, all services run as Docker containers defined in a single `docker-compose.yml` file on one shared network. This guarantees that every team member and every demo machine runs exactly the same versions of Kafka, PostgreSQL and Redis.

- **kafka** (with Zookeeper or KRaft) — message broker on port 9092.
- **producer** — the transaction simulator generating demo traffic.

- **api** — FastAPI and the Kafka consumer, with the trained model mounted as a file, on port 8000.
- **redis** — Redis 7 on port 6379.
- **postgres** — PostgreSQL 14 with a named volume *pgdata* so data survives restarts.
- **web** — Nginx serving `index.html`, `style.css`, `app.js` and the `Chart.js` bundle on port 80 and proxying `/api` to the api container.

CI/CD pipeline

A GitHub Actions workflow runs on every pull request: it installs dependencies, runs PyTest unit tests for feature engineering, scoring and API endpoints, and builds the Docker images. Only branches that pass are merged into *main*. The same Compose file can be deployed to a cloud virtual machine (for example, AWS EC2 or Azure VM); for larger scale, each service maps naturally onto managed equivalents such as a managed Kafka service, a managed PostgreSQL database and a managed Redis cache, as discussed in Chapter 11.

CHAPTER -5 DESIGN

5.1 ER Diagram / Database Schema

FraudShield — Entity Relationship Diagram

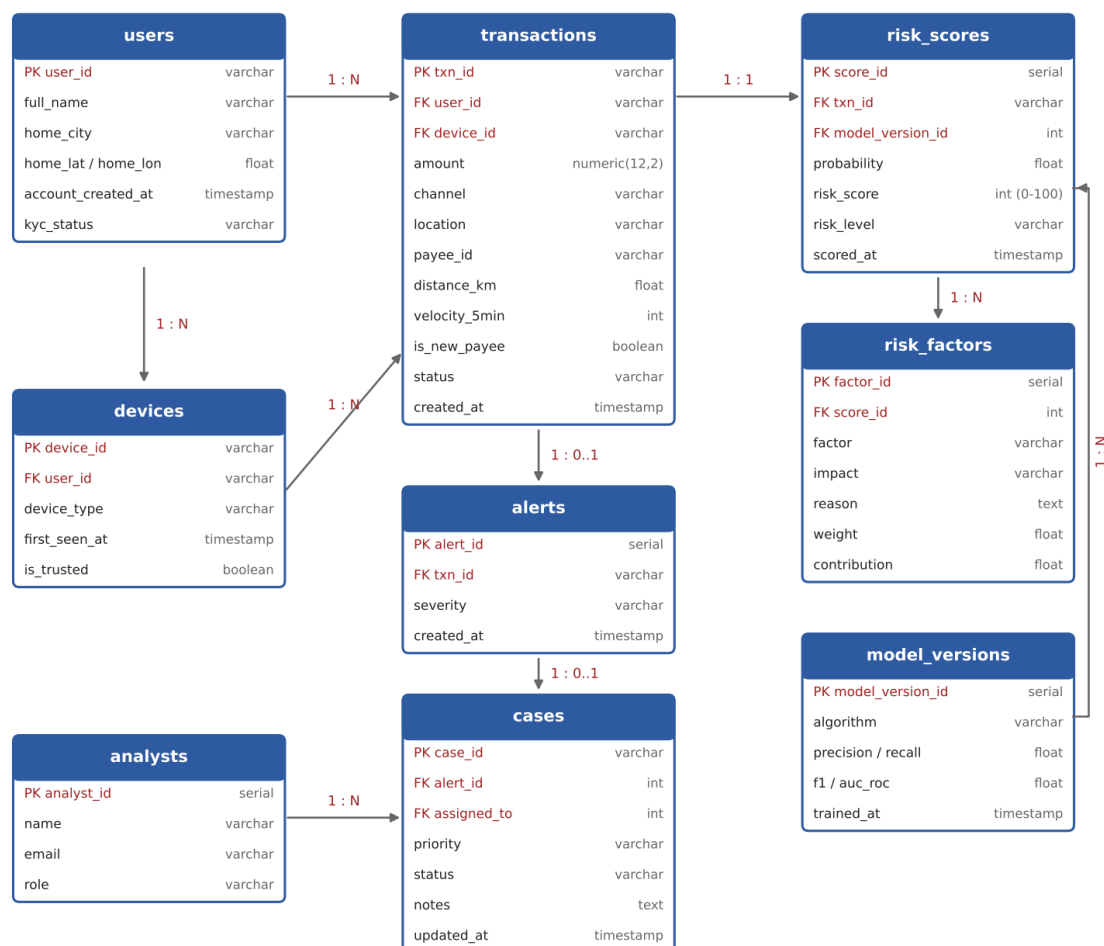


Figure 5.1 Entity relationship diagram of the FraudShield database

The schema is centred on the **transactions** table. Each user can own many devices and make many transactions. Each transaction receives exactly one risk score, and each score is broken down into several risk factors (one per feature contribution). A transaction whose score crosses the alert threshold generates an alert, an alert may be promoted into a case, and each case is assigned to one analyst. Every score references the model version that produced it.

Table 5.1 Main tables

Table	Key columns	Notes
users	user_id, home_city, home_lat/lon, account_created_at	Home location used for distance_km; creation date for account_age_days.

Table	Key columns	Notes
devices	device_id, user_id, first_seen_at, is_trusted	Supports the new_device feature.
transactions	txn_id, user_id, amount, channel, location, distance_km, velocity_5min, status	Status: Approved, Flagged or Under Review.
risk_scores	txn_id, probability, risk_score, risk_level, model_version_id	risk_score is an integer 0–100.
risk_factors	score_id, factor, impact, reason, weight, contribution	Feeds the Score Breakdown table and 'Why was this flagged?'.
alerts	txn_id, severity, created_at	Severity: High, Medium, Low.
cases	case_id, alert_id, priority, status, assigned_to, notes	Status: Open, In Review, Escalated, Closed.
analysts	analyst_id, name, email, role	Assignees for cases; roles such as Analyst or Super Admin.
model_versions	algorithm, precision, recall, f1, auc_roc, trained_at	Shown on the Models page.

5.2 Risk Scoring and Case Workflow Design

The model outputs a probability between 0 and 1. This is multiplied by 100 and rounded to give the risk score. Thresholds then map the score to a risk level and decide the action taken:

Table 5.2 Risk bands

Risk score	Risk level	Action
0 – 39	Low	Transaction approved; recorded for monitoring only.
40 – 69	Medium	Medium alert raised; transaction marked Under Review.
70 – 100	High	High alert raised; transaction Flagged and a case opened automatically with High priority.

The thresholds are configuration values rather than hard-coded constants, so they can be tuned to balance precision against recall based on the evaluation results and the team's tolerance for false positives.

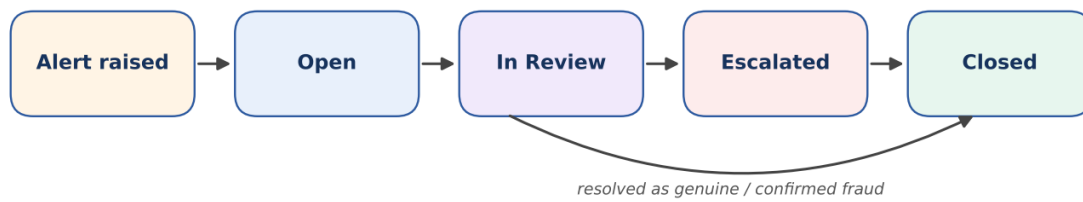


Figure 5.2 Case lifecycle in FraudShield

A case starts as **Open** when created from an alert. An analyst moves it to **In Review** while investigating, may **Escalate** it to a senior analyst for confirmed or complex fraud, and finally **Closes** it as either genuine or confirmed fraud. Every status change updates the *updated_at* timestamp, and the outcome can later be used as a feedback label for retraining.

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

The project was divided into three modules, each owned by one team member, with clearly defined interfaces between them.

Module	Owner	Responsibilities
Data Pipeline & ML Model	D. Rohini Tanmayi (Team Lead)	Feature engineering, class-imbalance handling, XGBoost training and tuning, RandomForest baseline, SHAP explanations.
Streaming, Backend API & Data Layer	S. Manvi Vaishnavi	Kafka ingestion, transaction simulator, FastAPI endpoints, Redis caching, PostgreSQL schema.
Dashboard & Explainability UI	Varshith	Fraud Risk Command Center: dashboard, live monitoring, transactions, alerts, cases, scoring form and model views.

Module 1: Data Pipeline and ML Model

Data preparation. Transaction records were cleaned, missing values handled, and categorical channel values encoded. The dataset was split into training, validation and test sets using stratified sampling so that the small fraction of fraud cases is represented proportionally in every split.

Feature engineering. The model uses behavioural features that capture how a transaction differs from the customer's normal pattern:

Table 6.1 Model features

Feature	Type	Why it matters
amount	Numeric	Very large or unusual amounts are a common fraud signal.
distance_km	Numeric	Distance between transaction location and the user's home; large jumps suggest stolen credentials.
velocity_5min	Integer	Number of transactions by the same user in the last 5 minutes; bursts indicate card testing or draining.
account_age_days	Integer	New accounts are more likely to be synthetic or mule accounts.
hour	Integer 0–23	Late-night activity is riskier for many customers.
new_device	Binary	First time this device has been seen for the user (account takeover signal).
new_payee	Binary	First payment to this beneficiary.
channel	Categorical	Mobile, Web, API or POS; each has a different risk profile.

Handling class imbalance. Because fraud is rare, a model that predicts “not fraud” for everything would still look highly accurate. Three strategies were compared: XGBoost's *scale_pos_weight* (ratio of negatives to positives), SMOTE oversampling, and random under-sampling. Model selection was based on F1-score and AUC-ROC on the validation set, not accuracy.

```
# ml/train.py (simplified)
neg, pos = (y_train == 0).sum(), (y_train == 1).sum()
model = XGBClassifier(
    n_estimators=400, max_depth=5, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8,
    scale_pos_weight=neg / pos, # imbalance handling
    eval_metric="aucpr", early_stopping_rounds=30)
model.fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=False)

proba = model.predict_proba(X_test)[: , 1]
print(classification_report(y_test, proba >= 0.5))
print("AUC-ROC:", roc_auc_score(y_test, proba))
model.save_model("models/xgb_fraud.json")
```

Explainability. A SHAP TreeExplainer is created from the trained model. For each transaction it returns one contribution per feature; positive values push the score towards fraud and negative values push it towards genuine. These are sorted and stored as risk factors.

Module 2: Streaming, Backend API and Data Layer

The simulator publishes JSON transactions to Kafka. The consumer enriches each event using Redis and passes it to the scoring function:

```
# backend/app/scoring.py (simplified)
def risk_level(score: int) -> str:
    return "High" if score >= 70 else "Medium" if score >= 40 else "Low"

def score_transaction(txn: TxnIn) -> ScoreOut:
    x = build_features(txn) # DataFrame with 1 row
    p = float(model.predict_proba(x)[0, 1])
    score = round(p * 100)
    shap_vals = explainer.shap_values(x)[0]
    factors = sorted(
        [{"factor": f, "contribution": float(v)} for f, v in zip(x.columns, shap_vals)],
        key=lambda d: abs(d["contribution"]), reverse=True)
    return ScoreOut(risk_score=score, risk_level=risk_level(score),
                    probability=p, factors=factors)
```

```
# backend/app/features.py (velocity from Redis)
def velocity_5min(r, user_id: str, ts: float) -> int:
    key = f"vel:{user_id}"
    r.zadd(key, {str(ts): ts})
    r.zremrangebyscore(key, 0, ts - 300) # drop events older than 5 min
    r.expire(key, 600)
    return r.zcard(key)
```

After scoring, the transaction, its score and risk factors are written to PostgreSQL in a single database transaction. If the risk level is Medium or High, an alert row is inserted; for High risk, a case is opened automatically. The result is also pushed to the Redis live-feed list and the alert counters are incremented.

Module 3: Dashboard and Explainability UI

The Fraud Risk Command Center was implemented as `index.html` (structure), `style.css` (layout, cards, colours) and `app.js` (data fetching, rendering and chart creation). Its views and logic are described in Section 6.2 and Chapter 7.

6.2 Front-end Logic (UI Rendering, State Management)

View switching

Every navigation button carries a `data-view` attribute. A single click handler removes the `active` class from all views and navigation items, adds it to the selected ones, updates the page title, and calls the render function for that view. Links inside cards (such as “View All Live Transactions →”) reuse the same mechanism.

```
// frontend/app.js (simplified)
document.querySelectorAll('[data-view]').forEach(btn =>
  btn.addEventListener('click', () => showView(btn.dataset.view)));

function showView(name) {
  document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
  document.querySelectorAll('.nav-item').forEach(n =>
    n.classList.toggle('active', n.dataset.view === name));
  document.getElementById('view-' + name).classList.add('active');
  renderers[name]?.(); // e.g. renderCases(), renderAlerts()
}
```

Rendering

Render functions fetch JSON and build HTML using template strings, injecting it into containers such as `#kpiGrid`, `#liveFeedList`, `#txnTable tbody` and `#casesTable tbody`. Risk scores are wrapped in coloured badges using the same Low/Medium/High thresholds as the backend. Charts are created once and then updated with new data, which avoids flicker and memory leaks.

State management

Application state is deliberately simple and kept in a small in-memory object: the currently selected transaction (for Live Monitoring), the current case-status filter, cached chart instances and the latest alert counts. The live feed and alert badge are refreshed on a polling interval. Selecting a transaction from the Transactions table stores its ID in state and switches to Live Monitoring, which then loads

the detail, gauge, “Why was this flagged?” list, score breakdown and SHAP chart.

6.3 API Integration

Table 6.2 REST endpoints

Method	Endpoint	Used by	Description
GET	/api/stats/dashboard	Dashboard	KPIs, risk distribution, fraud trend, top risk factors, alerts by channel.
GET	/api/transactions?limit=	Transactions, live feed	Latest transactions with risk score and status.
GET	/api/transactions/{txn_id}	Live Monitoring	Transaction info, score, risk level, factors and SHAP values.
POST	/api/score	Score Transaction	Scores a submitted transaction and returns score, level and explanation.
GET	/api/alerts	Alerts, bell badge	Alert counts by severity and flagged/under-review list.
GET	/api/cases?status=	Cases	Cases filtered by status, plus summary counts.
PATCH	/api/cases/{case_id}	Cases	Update status, priority, assignee or notes.
GET	/api/model/metrics	Dashboard, Models	Precision, recall, F1-score, AUC-ROC of the active model.
GET	/api/model/importances	Models	Global feature importances.
POST	/api/demo/reset	Models (Demo Controls)	Clears and re-seeds sample transactions and cases.

All responses are JSON. Requests are validated by Pydantic; for example, the scoring endpoint rejects an hour outside 0–23 or a negative amount with a 422 response that the front end shows to the user.

```
POST /api/score
{ "amount": 8950, "distance_km": 1200, "velocity_5min": 5,
  "account_age_days": 20, "hour": 0, "new_device": true,
  "new_payee": true, "channel": "Mobile", "location": "New York, USA" }

200 OK
{ "risk_score": 92, "risk_level": "High",
  "factors": [ {"factor": "amount", "impact": "High", "contribution": 0.31,
    "reason": "Amount far above user's typical spend"}, ... ] }
```

The response shown above is illustrative of the format; the exact score depends on the trained model.

6.4 Authentication and Authorisation

The dashboard identifies the signed-in analyst in the top bar together with their role (for example, Super Admin). Access control is designed around roles:

Role	Permissions
Analyst	View dashboard, transactions and alerts; work on assigned cases; score test transactions.
Senior Analyst	All Analyst permissions plus escalation handling and re-assignment of cases.
Super Admin	All permissions, including model information and the Reset Demo Data control.

On the API side, login issues a signed JWT access token containing the analyst ID and role; protected routes use a FastAPI dependency that verifies the token and checks the role before executing. Passwords are stored as bcrypt hashes. Destructive operations such as `/api/demo/reset` are restricted to Super Admin. CORS is limited to the dashboard's origin, and all SQL access goes through SQLAlchemy parameterised queries. For the academic demonstration a single seeded Super Admin account is used; full multi-user management is listed under future enhancements.

CHAPTER 7- FEATURES

7.1 List of Main Features

1. Real-time transaction ingestion through Kafka
2. Behavioural feature engineering with Redis
3. ML-based 0–100 risk scoring with Low / Medium / High levels
4. SHAP-based explanation (“Why was this flagged?”)
5. Command-center Dashboard with KPIs and charts
6. Live Transactions Feed
7. Transactions explorer
8. Live Monitoring (transaction detail and score breakdown)
9. Alerts by severity and channel
10. Case management workflow
11. Score Transaction (what-if scoring form)
12. Models page with performance and feature importances
13. Demo controls (reset and re-seed)
14. Global search and notification badge

7.2 How Each Feature Works

1. Real-time Transaction Ingestion

User perspective: new transactions appear on the dashboard within moments of being created, without refreshing the page.

Technical description: the simulator publishes JSON events to the *transactions.raw* Kafka topic. A consumer group reads the topic, so additional consumers can be added to scale throughput. Kafka retains events, which allows replay during testing.

2. Behavioural Feature Engineering

User perspective: analysts see meaningful context for each transaction — how far it is from home, how many transactions happened in the last five minutes, whether the device or payee is new.

Technical description: Redis sorted sets and sets compute velocity, new-device and new-payee flags in constant time. Distance is calculated with the haversine formula from the user's stored home

coordinates. Account age is derived from the account creation date.

3. ML-based Risk Scoring

User perspective: every transaction carries a single, easy-to-read risk score out of 100, coloured green, amber or red.

Technical description: the XGBoost model returns a fraud probability that is scaled to 0–100 and mapped to a level using configurable thresholds (Table 5.2). The score and the model version are stored with the transaction.

4. Explainability (“Why was this flagged?”)

User perspective: on the Live Monitoring page, analysts see a plain-language list of reasons, a Score Breakdown table with *Factor*, *Impact*, *Reason*, *Weight* and *Contribution* columns, and a bar chart of feature contributions.

Technical description: SHAP values from TreeExplainer are sorted by absolute size. Each factor is given an impact level and a templated reason, for example “velocity_5min = 5: five transactions in five minutes, far above normal”. The contributions are plotted as a horizontal Chart.js bar chart, positive values in red and negative values in green.

5. Command-center Dashboard

User perspective: the landing page shows KPI cards at the top, followed by Risk Score Distribution, Fraud Trend, Top Risk Factors, the Live Transactions Feed, Alerts Overview, Alerts by Channel and Model Performance.

Technical description: a single call to `/api/stats/dashboard` returns pre-aggregated data computed with SQL GROUP BY queries; the front end injects KPI cards into `#kpiGrid` and draws doughnut, line and bar charts.

6. Live Transactions Feed

User perspective: a scrolling list marked ● LIVE shows the most recent transactions with amount, channel and risk badge, and a link to view all transactions.

Technical description: the feed reads the capped Redis list `feed:live`, so it is fast even when PostgreSQL holds many rows.

7. Transactions Explorer

User perspective: a table with Txn ID, User, Amount, Channel, Location, Risk Score, Status and Time, with a Refresh button. Clicking a row opens the transaction in Live Monitoring.

Technical description: data comes from `/api/transactions` ordered by time, with a limit for performance.

8. Live Monitoring (Transaction Detail)

User perspective: three cards show Transaction Information, a Risk Score gauge (score/100 with level text) and “Why was this flagged?”, followed by the Score Breakdown table and the Model Explanation chart. If nothing is selected, quick-pick buttons offer recent transactions.

Technical description: `/api/transactions/{id}` returns the transaction joined with its score and risk factors. The gauge is a half doughnut drawn on a 180×180 canvas.

9. Alerts

User perspective: large counters for High, Medium and Low risk alerts in red, amber and green, and a table of flagged and under-review transactions. The bell in the top bar shows the open alert count.

Technical description: counts are read from the alerts table (and cached in Redis for the badge); the list joins alerts with transactions.

10. Case Management

User perspective: summary tiles at the top, a status filter (All, Open, In Review, Escalated, Closed) and a table with Case ID, Priority, Status, User, Amount, Risk Score, Assigned To and an Action button to move the case forward.

Technical description: `GET /api/cases?status=` filters on the server; actions call `PATCH /api/cases/{id}`, which validates allowed status transitions and updates `updated_at`.

11. Score Transaction

User perspective: a form with Amount, Distance from home, Velocity in last 5 minutes, Account age, Hour of day, New device, New payee, Channel and Location. It is pre-filled with a risky example (amount 8,950, 1,200 km from home, 5 transactions in 5 minutes, 20-day-old account, midnight, new device and payee). Submitting shows the score and explanation instantly.

Technical description: the form posts JSON to `/api/score`; the same scoring function as the streaming path is used, so results are consistent. This feature is especially useful for demonstrations

and for analysts testing “what-if” scenarios.

12. Models Page

User perspective: model performance metrics and a Feature Importances chart show how the model behaves overall.

Technical description: metrics come from the *model_versions* table; importances are mean absolute SHAP values computed on the test set.

13. Demo Controls

User perspective: a Reset Demo Data button clears and re-seeds the database with a fresh batch of sample transactions and cases — useful before a live presentation.

Technical description: *POST /api/demo/reset* truncates operational tables, clears related Redis keys and replays a seeded batch through the scoring function. It is restricted to the Super Admin role.

14. Global Search and Notifications

User perspective: the search box in the top bar finds transactions, users and cases by ID or name; the bell badge always shows the current number of alerts.

Technical description: search uses indexed ILIKE queries across the main tables; the badge is refreshed together with the live feed.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Postman was used to test every FastAPI endpoint independently of the dashboard. A shared Postman collection named *FraudShield API* was kept in the repository with an environment variable `{{base_url}}` so that tests could be run against a local or deployed backend.

- **Positive tests:** valid requests to `/api/score`, `/api/transactions`, `/api/alerts` and `/api/cases` return 200 with the expected JSON fields.
- **Validation tests:** negative amounts, an hour of 25, a missing channel or an unknown channel value return 422 with a clear error message.
- **Not-found tests:** unknown transaction or case IDs return 404.
- **State-transition tests:** PATCH requests that try to reopen a Closed case or skip from Open straight to Closed without review are rejected.
- **Authorisation tests:** calling `/api/demo/reset` without a Super Admin token returns 401/403.
- **Response-time checks:** Postman test scripts assert that `/api/score` responds within an agreed limit on the demo machine.

```
// Postman test script for POST /api/score
pm.test("status 200", () => pm.response.to.have.status(200));
const b = pm.response.json();
pm.test("score in range", () => pm.expect(b.risk_score).to.be.within(0, 100));
pm.test("level valid", () =>
  pm.expect(["Low", "Medium", "High"]).to.include(b.risk_level));
pm.test("has factors", () => pm.expect(b.factors.length).to.be.above(0));
```

On the front end, each view was tested manually in Chrome and Firefox with DevTools open to confirm that the correct API calls were made, charts rendered without console errors, and the layout stayed usable at common screen sizes.

8.2 Integration Testing

Integration tests verified that the components work correctly together, from the moment an event is published to the moment it appears in the dashboard.

1. **Kafka → consumer:** a test event published to *transactions.raw* is consumed exactly once and appears in *transactions.scored*.
2. **Consumer → Redis:** sending six events for one user within five minutes produces velocity values 1 to 6; events older than 300 seconds are excluded.

3. **Scoring** → **PostgreSQL**: a scored transaction creates one row in transactions, one in risk_scores and one row per feature in risk_factors.
4. **Alert and case creation**: a High-risk transaction creates an alert and an Open case with High priority; a Low-risk one creates neither.
5. **API** → **dashboard**: the new transaction appears in the live feed, the bell badge increments, and the case appears under the Open filter.
6. **Consistency**: the same input scored through Kafka and through the Score Transaction form returns the same risk score.
7. **Reset**: Reset Demo Data empties and re-seeds tables and Redis keys, and all views reload without errors.

These tests were run inside the Docker Compose environment so that versions of Kafka, Redis and PostgreSQL matched the demo setup exactly.

8.3 Tools Used for Testing

Tool	Purpose
Postman	Manual and scripted API tests; shared collection and environments.
PyTest	Unit tests for feature functions (velocity, distance, risk_level), scoring and API routes via FastAPI TestClient.
scikit-learn metrics	Model evaluation: classification report, confusion matrix, ROC-AUC, precision-recall curve.
Swagger UI	Quick exploratory calls during development.
Browser DevTools	Network, console and layout debugging of the dashboard.
pgAdmin / redis-cli	Verifying stored rows and Redis keys after integration tests.
Docker Compose	Reproducible environment for integration testing.
GitHub Actions	Running PyTest automatically on each pull request.

8.4 Test Cases and Results

Table 8.1 Functional test cases

ID	Test case	Expected result	Status
TC01	Load dashboard	KPI cards, all charts, live feed and alerts render without errors	Pass
TC02	Navigate between all seven views	Correct view and title shown; active nav item highlighted	Pass

ID	Test case	Expected result	Status
TC03	Score low-risk transaction (small amount, home city, known device, daytime)	Low risk level, green badge	Pass
TC04	Score the default risky example on Score Transaction form	High risk level with ranked reasons	Pass
TC05	Submit form with hour = 25	Validation error shown; no score produced	Pass
TC06	Six transactions for one user within 5 minutes	velocity_5min increases and risk score rises	Pass
TC07	Click a transaction in Transactions table	Live Monitoring opens with gauge, reasons, breakdown and SHAP chart	Pass
TC08	High-risk transaction arrives	Alert created, bell badge increments, Open case created	Pass
TC09	Filter cases by 'Escalated'	Only escalated cases listed	Pass
TC10	Move case Open → In Review → Closed	Status updates persist after refresh	Pass
TC11	Reset Demo Data as Super Admin	Fresh sample data loaded; status message shown	Pass
TC12	Call /api/demo/reset without admin token	401/403 returned; data unchanged	Pass
TC13	Stop Redis container temporarily	API returns clear error; recovers when Redis restarts	Pass
TC14	Search for an existing transaction ID	Matching transaction shown	Pass

Model evaluation

The model was evaluated on a held-out, stratified test set that was never used during training or tuning. Because fraud is rare, accuracy alone is not reported as a success measure; instead the following metrics are used:

- **Precision** — of the transactions flagged as fraud, how many were actually fraud (controls false positives).
- **Recall** — of all fraudulent transactions, how many were caught (controls missed fraud).
- **F1-score** — the harmonic mean of precision and recall.
- **AUC-ROC** — how well the model ranks fraudulent transactions above genuine ones across all thresholds.

Table 8.2 Model comparison on the test set

Model	Precision	Recall	F1-score	AUC-ROC
RandomForest (baseline)				
XGBoost + scale_pos_weight				
XGBoost + SMOTE				

Note: the highlighted cells should be filled with the values from the final training run (the same figures displayed on the Models page).

Overall, the functional tests confirmed that the pipeline, API and dashboard behave as designed, and the integration tests confirmed that a transaction flows correctly from Kafka all the way to an analyst's case list.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

FraudShield is deployed with Docker Compose, which starts every service with one command. The steps below take a fresh machine to a running system.

1. **Install prerequisites:** Git, Docker Desktop (or Docker Engine with the Compose plugin) and, for training, Python 3.10+.
2. **Clone the repository:** *git clone* the FraudShield repository and change into the project folder.
3. **Create the environment file:** copy *.env.example* to *.env* and set database credentials, JWT secret and thresholds (Section 9.2).
4. **Train or copy the model:** run *python ml/train.py* to produce *models/xgb_fraud.json*, or use the committed model file.
5. **Start the stack:** run *docker compose up -d --build*. Compose pulls Kafka, PostgreSQL, Redis and Nginx images and builds the api and producer images.
6. **Initialise the database:** on first start the api container creates tables through SQLAlchemy and seeds the Super Admin account and sample data.
7. **Verify health:** open *http://localhost:8000/docs* for the API and *http://localhost* for the dashboard; check *docker compose ps* shows all services healthy.
8. **Start demo traffic:** the producer container begins publishing transactions; they appear in the live feed within seconds.

```
git clone https://github.com/<team>/fraudshield.git
cd fraudshield
cp .env.example .env
python ml/train.py                # optional: retrain the model
docker compose up -d --build
docker compose ps                 # all services should be 'running'
docker compose logs -f api        # watch scoring logs
```

9.2 Environment Configuration

All configuration is supplied through environment variables so that the same images can run on a laptop or in the cloud without code changes. Secrets are never committed; *.env* is listed in *.gitignore*.

Table 9.1 Environment variables

Variable	Example	Description
KAFKA_BOOTSTRAP	kafka:9092	Kafka broker address
KAFKA_TOPIC_RAW	transactions.raw	Input topic
KAFKA_TOPIC_SCORED	transactions.scored	Output topic
DATABASE_URL	postgresql://fs:****@postgres:5432/fraudshield	PostgreSQL connection string
REDIS_URL	redis://redis:6379/0	Redis connection
MODEL_PATH	/models/xgb_fraud.json	Trained model file
RISK_MEDIUM / RISK_HIGH	40 / 70	Risk band thresholds
JWT_SECRET	(random 32+ chars)	Signing key for access tokens
CORS_ORIGINS	http://localhost	Allowed dashboard origin
SIM_TPS	5	Simulator transactions per second

```
# docker-compose.yml (excerpt)
services:
  api:
    build: ./backend
    env_file: .env
    ports: ["8000:8000"]
    volumes: [".:/ml/models:/models:ro"]
    depends_on: [kafka, postgres, redis]
  postgres:
    image: postgres:14
    environment: { POSTGRES_DB: fraudshield, POSTGRES_USER: fs, POSTGRES_PASSWORD: ${PG_PASSWORD} }
    volumes: ["pgdata:/var/lib/postgresql/data"]
  redis:
    image: redis:7
  web:
    image: nginx:alpine
    volumes: [".:/frontend:/usr/share/nginx/html:ro", "./nginx.conf:/etc/nginx/conf.d/default.conf:ro"]
    ports: ["80:80"]
  volumes: { pgdata: {} }
```

9.3 Hosting of Frontend and Backend

Frontend

The Fraud Risk Command Center consists only of static files (index.html, style.css, app.js and the Chart.js bundle). They are served by Nginx, which also acts as a reverse proxy: requests to `/api/` are forwarded to the FastAPI container. Serving the dashboard and the API from the same origin avoids CORS problems in production. The static files could equally be hosted on GitHub Pages, Netlify or an S3 bucket with the API URL configured in app.js.

Backend

The FastAPI application runs under Uvicorn inside its container, with the Kafka consumer running as a background task in the same image (or as a separate container for scaling). For a cloud deployment, the whole Compose stack can run on a single virtual machine such as AWS EC2 or Azure VM with ports 80 and 443 open and HTTPS provided by Nginx with a Let's Encrypt certificate. For larger workloads, the services map onto managed offerings: a managed Kafka service (e.g., Amazon MSK or Confluent Cloud), a managed PostgreSQL database (e.g., Amazon RDS) and a managed Redis cache (e.g., Amazon ElastiCache), with the API deployed on a container service.

For the academic review and demonstrations, FraudShield was hosted locally using Docker Compose, with the Reset Demo Data control used before each presentation to start from a clean, consistent dataset.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

1. Severe class imbalance

Fraudulent transactions form a very small fraction of the data. Early models reached very high accuracy simply by predicting every transaction as genuine, while catching almost no fraud.

2. Training/serving feature mismatch

Features computed in the training notebook (for example, velocity from historical data with pandas) did not initially match features computed in real time from Redis, which caused the live model to behave differently from the offline evaluation.

3. Real-time state for velocity features

Counting a user's transactions in a sliding five-minute window by querying PostgreSQL for every event was too slow and placed heavy load on the database.

4. Making SHAP output understandable

Raw SHAP values (for example, +0.27 for *velocity_5min*) mean little to a fraud analyst. Presenting them directly in the dashboard confused early testers.

5. Running Kafka locally

Kafka's listener configuration inside Docker was difficult to get right; the API container and host machine needed different advertised addresses, which caused connection failures.

6. Keeping the dashboard responsive

Re-creating Chart.js charts on every refresh caused flicker and growing memory usage in long demo sessions.

7. Coordinating three modules

With three team members working in parallel on ML, backend and frontend, changes to JSON field names occasionally broke other modules.

10.2 Solutions Applied

Issue	Solution
Class imbalance	Used stratified splits, <code>scale_pos_weight</code> in XGBoost, compared SMOTE and under-sampling, and selected models on F1-score and AUC-ROC instead of accuracy. Threshold tuning on the validation set balanced precision and recall.
Feature mismatch	Moved feature logic into a shared <i>features.py</i> module imported by both the training script and the scoring service, with unit tests asserting identical output.
Velocity state	Implemented sliding-window counters with Redis sorted sets (ZADD, ZREMRANGEBYSCORE, ZCARD) and key expiry, giving constant-time lookups.
Understandable SHAP	Mapped each feature to a reason template and an impact level (High/Medium/Low) and displayed the top factors as plain sentences plus a colour-coded bar chart.
Kafka in Docker	Configured separate internal and external listeners (kafka:9092 inside the network, localhost:29092 outside) and added health checks so the API waits for Kafka.
Dashboard performance	Created each chart once and updated its data in place; capped the live feed list length; used a modest polling interval.
Module coordination	Agreed API contracts as Pydantic schemas first, documented them in Swagger UI, and used pull-request reviews and a shared Postman collection.

10.3 Current Limitations or Known Bugs

- **Simulated and public data:** the model is trained on synthetic/public data, which cannot capture every real-world fraud pattern or customer behaviour.
- **No automatic retraining:** analyst decisions on closed cases are stored but not yet fed back into model retraining.
- **Single-transaction view:** features describe one user's behaviour; coordinated fraud rings spanning many accounts or devices are not detected.
- **Polling instead of push:** the dashboard refreshes by polling; under heavy load, updates may lag by a few seconds. WebSockets would be more efficient.
- **Basic access control:** role-based access is implemented for core actions, but full user management, audit logs and SSO are not yet available.
- **Fixed thresholds:** risk bands are global; they are not yet tuned per channel or customer segment.

- **Model metric label:** the dashboard's Model Performance card displays the RandomForest baseline label; it should be switched to show the active XGBoost model.
- **Single-node deployment:** the Compose setup is not highly available; a failure of the host stops all services.
- **No alert notifications outside the dashboard:** analysts must be looking at the dashboard to see new alerts.

These limitations are understood and are addressed in the future enhancements described in the next chapter.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

1. Feedback-driven model retraining

Closed cases carry a confirmed outcome — genuine or fraud. A scheduled retraining pipeline will collect these analyst-confirmed labels, retrain the model, evaluate it against the current version, and promote it only if precision, recall and AUC-ROC improve. The *model_versions* table already supports tracking each version.

2. Graph-based fraud-ring detection

Fraud rings share devices, IP addresses, payees and addresses across many accounts. Building a graph of these links and adding features such as “number of accounts sharing this device” or community-detection scores will allow FraudShield to detect coordinated attacks that look normal at the single-account level.

3. Multi-channel alerting

High-risk alerts will be sent to analysts through email, SMS and Slack, using a separate notification service that consumes the *transactions.scored* Kafka topic, so analysts do not need to watch the dashboard continuously.

4. Real-time push updates

Replacing polling with WebSockets or Server-Sent Events will push new transactions and alerts to the dashboard instantly and reduce server load.

5. Richer case management

Planned additions include case notes with attachments, a full audit trail of every status change, SLA timers for High-priority cases, and bulk actions.

6. Adaptive thresholds

Risk thresholds will be tuned per channel and customer segment, and a cost-based threshold optimiser will choose the threshold that minimises expected loss from both fraud and false positives.

7. Model monitoring and drift detection

Tracking the distribution of input features and scores over time will reveal data drift, triggering alerts or retraining before performance degrades.

11.2 Possible Integrations or Optimisations

Area	Integration / optimisation
Cloud scale	Deploy on Kubernetes with horizontal autoscaling of the scoring consumers based on Kafka consumer lag.
Managed services	Use managed Kafka, PostgreSQL and Redis offerings to improve availability and reduce maintenance.
Payment integration	Expose the scoring API as a pre-authorisation hook for payment gateways so high-risk payments can be held before completion.
Device and IP intelligence	Integrate device fingerprinting and IP reputation services as additional features.
Identity	Single sign-on (OAuth2/OpenID Connect) for analysts, with fine-grained roles and audit logging.
Model serving	Export the model to ONNX or use a dedicated model server to reduce latency and support A/B testing of model versions.
Analytics	Stream scored transactions into a data warehouse for long-term trend analysis and reporting.
Front end	Migrate the dashboard to a component framework such as React for larger teams, keeping the same API contracts.
Mobile	A mobile companion app for supervisors to approve escalations on the go.

These enhancements would move FraudShield from an academic demonstration towards a production-ready fraud operations platform while keeping its core design unchanged.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

FraudShield set out to show that fraud detection can be accurate, real-time and explainable, and that monitoring, alerting and investigation can live in one place. The project began from a clear problem: rule-based systems are rigid, produce many false positives, cannot explain themselves, and are surrounded by disconnected tools.

The resulting platform ingests transactions through Apache Kafka, computes behavioural features with Redis, scores each transaction with an imbalance-aware XGBoost model, explains every score with SHAP, stores durable records in PostgreSQL, and presents everything in the Fraud Risk Command Center. Analysts can watch the live feed, open any transaction to see why it was flagged, triage alerts by severity and channel, manage cases through a defined lifecycle, test hypothetical transactions, and review model performance.

The work was divided across three modules — the ML pipeline, the streaming backend and data layer, and the dashboard — and integrated through well-defined JSON APIs. The complete system runs with a single Docker Compose command, which made it reliable to demonstrate and easy for others to reproduce.

12.2 What Was Achieved

- An end-to-end, industry-style fraud detection pipeline from real-time ingestion to analyst decision.
- An XGBoost model trained with explicit handling of class imbalance and evaluated on precision, recall, F1-score and AUC-ROC, with a RandomForest baseline for comparison.
- Real-time behavioural features (velocity, distance, new device/payee, account age, hour, channel) computed in constant time with Redis.
- A 0–100 risk score with Low/Medium/High bands and automatic alert and case creation.
- Per-transaction explanations through a “Why was this flagged?” list, a score breakdown table and a feature-contribution chart.
- A seven-view Fraud Risk Command Center: Dashboard, Live Monitoring, Transactions, Alerts, Cases, Score Transaction and Models.
- A unified case-management workflow with priorities, statuses and assignment.
- Containerised deployment, automated tests and demo controls for reliable presentations.

Most importantly, the project demonstrates that explainability and usability are not extras: they are what turns a model's prediction into a decision an analyst can trust.

12.3 Skills Learned During Development

Technical skills

- **Machine learning on imbalanced data:** stratified splitting, class weighting, SMOTE, threshold tuning, and choosing appropriate metrics.
- **Explainable AI:** computing and interpreting SHAP values and turning them into language that non-technical users understand.
- **Stream processing:** Kafka topics, producers, consumer groups and configuring Kafka in Docker.
- **Backend engineering:** designing REST APIs in FastAPI, request validation with Pydantic, ORM modelling with SQLAlchemy.
- **Data modelling:** designing a normalised relational schema and choosing what belongs in Redis versus PostgreSQL.
- **Front-end development:** building a multi-view dashboard in HTML, CSS and JavaScript and visualising data with Chart.js.
- **DevOps:** Docker Compose, environment-based configuration, GitHub Actions and Git branching workflows.
- **Testing:** unit tests with PyTest, API tests with Postman and end-to-end integration testing.

Professional skills

Working as a three-member team taught us to agree on interfaces early, review each other's code, communicate changes clearly and plan work around reviews and demonstrations. Preparing for presentations improved our ability to explain a complex technical system to an audience with different levels of expertise. Above all, FraudShield showed us how the concepts from our coursework — machine learning, databases, networks and web development — come together to solve a real and important problem.

CHAPTER -13 REFERENCES

Research papers and books

1. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
2. Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*.
3. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
4. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
5. Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating Probability with Undersampling for Unbalanced Classification. *IEEE Symposium Series on Computational Intelligence*.
6. Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A Distributed Messaging System for Log Processing. *Proceedings of the NetDB Workshop*.
7. Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
8. Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.

Documentation and online resources

1. Apache Kafka Documentation. Retrieved from <https://kafka.apache.org/documentation/>
2. FastAPI Documentation. Retrieved from <https://fastapi.tiangolo.com/>
3. XGBoost Documentation. Retrieved from <https://xgboost.readthedocs.io/>
4. SHAP Documentation. Retrieved from <https://shap.readthedocs.io/>
5. scikit-learn User Guide. Retrieved from https://scikit-learn.org/stable/user_guide.html
6. imbalanced-learn Documentation. Retrieved from <https://imbalanced-learn.org/>
7. PostgreSQL 14 Documentation. Retrieved from <https://www.postgresql.org/docs/14/>
8. Redis Documentation. Retrieved from <https://redis.io/docs/>
9. SQLAlchemy Documentation. Retrieved from <https://docs.sqlalchemy.org/>
10. Chart.js Documentation. Retrieved from <https://www.chartjs.org/docs/>
11. Docker Compose Documentation. Retrieved from <https://docs.docker.com/compose/>

12. Postman Learning Center. Retrieved from <https://learning.postman.com/>
13. Credit Card Fraud Detection dataset (ULB Machine Learning Group). Kaggle. Retrieved from <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>

CHAPTER -14 APPENDICES

Screenshots of the app

[Insert screenshot: Dashboard — KPIs, Risk Score Distribution, Fraud Trend, Live Feed]

[Insert screenshot: Live Monitoring — Risk gauge, Why was this flagged?, Score Breakdown, SHAP chart]

[Insert screenshot: Alerts and Cases views]

[Insert screenshot: Score Transaction form with result, and Models page]

Sample code snippets

Kafka consumer loop (backend/consumer.py)

```
consumer = KafkaConsumer(TOPIC_RAW, bootstrap_servers=KAFKA_BOOTSTRAP,
                          group_id="scorers", value_deserializer=json.loads)
producer = KafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP,
                          value_serializer=lambda v: json.dumps(v).encode())

for msg in consumer:
    txn = TxnIn(**msg.value)
    result = score_transaction(txn)
    save_to_postgres(txn, result)          # transaction, score, factors, alert/case
    push_live_feed(redis, txn, result)
    producer.send(TOPIC_SCORED, {**msg.value, **result.dict()})
```

FastAPI scoring endpoint (backend/app/routers/score.py)

```
class TxnIn(BaseModel):
    amount: confloat(ge=0)
    distance_km: confloat(ge=0)
    velocity_5min: conint(ge=0)
    account_age_days: conint(ge=0)
    hour: conint(ge=0, le=23)
    new_device: bool = False
    new_payee: bool = False
    channel: Literal["Mobile", "Web", "API", "POS"]
    location: str | None = None

@router.post("/api/score", response_model=ScoreOut)
def score(txn: TxnIn):
    return score_transaction(txn)
```

Dashboard: rendering the risk gauge (frontend/app.js)

```
function drawGauge(score) {
    const color = score >= 70 ? '#e5484d' : score >= 40 ? '#f5a524' : '#30a46c';
    gauge?.destroy();
    gauge = new Chart(document.getElementById('riskGauge'), {
        type: 'doughnut',
        data: { datasets: [{ data: [score, 100 - score],
                               backgroundColor: [color, '#e9ecef'], borderWidth: 0 }] },
        options: { rotation: -90, circumference: 180, cutout: '75%',
                  plugins: { legend: { display: false }, tooltip: { enabled: false } } }
    });
    document.getElementById('riskScoreNum').textContent = score;
}
```

Installation/setup instructions

1. Install Git, Docker Desktop and Python 3.10 or later.
2. Clone the repository and open the project folder.
3. Copy *.env.example* to *.env* and set PG_PASSWORD and JWT_SECRET.

4. (Optional) Create a virtual environment, run `pip install -r ml/requirements.txt`, then `python ml/train.py` to retrain the model.
5. Run `docker compose up -d --build` and wait until all containers are running.
6. Open `http://localhost` in a browser to use the Fraud Risk Command Center, and `http://localhost:8000/docs` to explore the API.
7. Sign in with the seeded Super Admin account and, if needed, press Reset Demo Data on the Models page to load fresh sample data.

User manual or guide

Dashboard. The landing page. Read the KPI cards for today's totals, check Risk Score Distribution and Fraud Trend for the overall picture, see which factors drive risk most in Top Risk Factors, and watch the Live Transactions Feed. Click “View All Live Transactions” to open the full list.

Transactions. Browse recent transactions with their risk score and status. Press Refresh to reload. Click any row to open its full analysis.

Live Monitoring. Shows one transaction in detail: its information, the risk gauge, the reasons it was flagged, a score breakdown table and the model explanation chart. Use the quick-pick buttons if no transaction is selected.

Alerts. See how many High, Medium and Low alerts exist and review the list of flagged and under-review transactions. The bell icon in the top bar shows the same count.

Cases. Use the status filter to focus on Open, In Review, Escalated or Closed cases. Use the Action button to move a case to the next stage once it has been investigated.

Score Transaction. Enter or adjust the amount, distance from home, velocity, account age, hour, new device, new payee, channel and location, then press Score Transaction to see the risk score and explanation. Try changing one field at a time to see how each factor affects the score.

Models. Review model performance and feature importances. Super Admins can press Reset Demo Data to restore a clean demo dataset before a presentation.

Geo Tag photos with guide

[Insert geo-tagged photo of the team with the guide]

Review forms with guide signatures

[Insert scanned Review-1 rubrics and evaluation form with guide signature]

